

Exercises on Coreset Technique and Clustering

Exercise 1 Show that the L_1 (Euclidean) distance satisfy the four requirements for a metric space.

Solution. Recall that given a set M with distance function $d(\cdot)$, (M, d) is a *metric space* if the following conditions hold for any $x, y, z \in M$:

1. $d(x, y) \geq 0$;
2. $d(x, y) = 0$ if and only if $x = y$;
3. $d(x, y) = d(y, x)$; (symmetry)
4. $d(x, z) \leq d(x, y) + d(y, z)$; (triangle inequality)

Recall that the L_1 distance between two points $x, y \in \mathbb{R}^n$ is

$$d_{L1}(x, y) = \sum_{i=1}^n |x_i - y_i|,$$

where the x_i 's and y_i 's denote the coordinates. We observe that for any three real numbers a, b and c , we have $|a - b| \leq |a - c| + |c - b|$, which can be easily proved considering all possible relative positions of a, b and c on the line. Now we prove that the four conditions above are satisfied.

1. $d_{L1}(x, y) \geq 0$. It follows since all terms of the summation are ≥ 0 .
2. $d_{L1}(x, y) = 0$ if and only if $x = y$.
 Proof of the "if" direction: if $x = y$, we then clearly have $d_{L1}(x, y) = \sum_{i=1}^n |x_i - y_i| = 0$ since $|x_i - y_i| = 0$ for each $i \geq 1$.
 Proof of the "only if" direction: let $d_{L1}(x, y) = 0$ and assume by contradiction that $x \neq y$ (say that $x_1 \neq y_1$). Then $d_{L1}(x, y) = \sum_{i=1}^n |x_i - y_i| > 0$ since $|x_1 - y_1| > 0$ and $|x_i - y_i| \geq 0$ if $i \geq 2$. We have a contradiction, and hence $x = y$.
3. $d_{L1}(x, y) = d_{L1}(y, x)$. It follows since $|a - b| = |b - a|$ for any two reals a and b .
4. $d(x, z) \leq d(x, y) + d(y, z)$. By the initial observation, we have that

$$\begin{aligned} d_{L1}(x, z) &= \sum_{i=1}^n |x_i - z_i| \\ &= \sum_{i=1}^n (|x_i - y_i + y_i - z_i|) \end{aligned}$$

$$\begin{aligned}
&\stackrel{(1)}{\leq} \sum_{i=1}^n (|x_i - y_i| + |y_i - z_i|) \\
&= \left(\sum_{i=1}^n |x_i - y_i| \right) + \left(\sum_{i=1}^n |y_i - z_i| \right) \\
&= d_{L1}(x, y) + d_{L1}(y, z).
\end{aligned}$$

In the step denoted with (1) we used the following inequality: $|a + b| \leq |a| + |b|$ for any $a, b \in \mathbb{R}$ (the inequality follows since $-|a| - |b| \leq a + b \leq |a| + |b|$).

□

Exercise 2 Consider a pointset P of N points represented as set of key-value pairs (ID_x, x) , where ID_x is a distinct integer in $[0, N)$, and x is a point. Let also S be a set of k centers. Considering S as a global variable, show the assignment of each point to the closest center can be performed in MapReduce in 1 round using $M_L = O(k)$.

Solution. We prescribe that for each input point (ID_x, x) the output pair $(ID_x, (x, c(x)))$ is returned, where $c(x) \in S$ is the center closest to x (with ties broken arbitrarily). We first consider the following straightforward algorithm.

Round 1

- *Map phase*: map each pair (ID_x, x) into $(ID_x, (x, c(x) = \operatorname{argmin}_{c \in S} d(x, c)))$.
- *Reduce phase*: empty.

We have that $M_L = \Theta(k)$, since each application of the map function requires space $\Theta(k)$ for storing a copy of S .

IMPORTANT OBSERVATION. If we ignore the space occupied by the copies of S , the aggregate space M_A is clearly linear in N . In practice, one copy of S is stored at each worker executing the map phase (for example, in Spark this is the case if S is defined as a *broadcast variable*), and if the number of workers is $O(N/k)$, which is a reasonable assumption in most practical scenarios, the aggregate space remains linear in N . □

Exercise 3 Show that the *Farthest-First Traversal* algorithm can be implemented to run in $O(N \cdot k)$ time.

Solution. With each point p we maintain a value called curr-dist_p which, immediately after center c_i is selected, contains the distance between p and the closest center among $c_1, c_2, \dots, c_{i-1}$. Initially, after the arbitrary selection of c_1 , curr-dist_p is set equal to null. It is easy to see that, for every $i \geq 1$, if the property holds after c_i is selected, one scan of the pointset P ($O(N)$ time) is sufficient to update the value curr-dist_p with the distance between p and the closest center among $c_1, c_2, \dots, c_i$, and to select the next center c_{i+1} as the point p with minimum curr-dist_p . This immediately implies that Farthest-First Traversal can be implemented in $O(N \cdot k)$ time. □

Exercise 4 Let P be a set of N points in a metric space (M, d) , and let $T \subseteq P$ be a coreset of $|T| > k$ points such that for each $x \in P$ we have $d(x, T) \leq \epsilon \Phi_{\text{kcenter}}^{\text{opt}}(P, k)$, for some $\epsilon \in (0, 1)$. Let S be the set of k centers obtained by running the Farthest-First Traversal algorithm on T . Prove an upper bound to $\Phi_{\text{kcenter}}(P, S)$ as a function of ϵ and $\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$.

Solution. By the hypothesis, we know that: (A) for each $x \in P$ there exists $y \in T$ such that $d(x, y) \leq \epsilon \Phi_{\text{kcenter}}^{\text{opt}}(P, k)$. We now show that: (B) for each $y \in T$ there exists $c \in S$ such that $d(y, c) \leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$. By combining (A) and (B) using triangle inequality, we immediately obtain that for each $x \in P$ there exists $c \in S$ such that $d(x, c) \leq (2 + \epsilon)\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$, which yields

$$\Phi_{\text{kcenter}}(P, S) \leq (2 + \epsilon)\Phi_{\text{kcenter}}^{\text{opt}}(P, k).$$

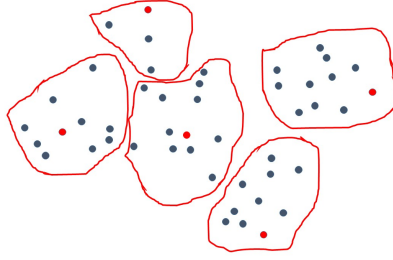
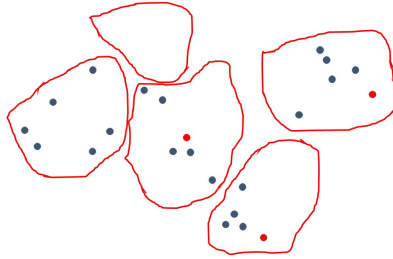
In order to show (B) we mimic the analysis of MR-Farthest First Traversal. Let $S = \{c_1, c_2, \dots, c_k\}$, where c_i is the i -th center discovered when running Farthest-First Traversal algorithm on T , and let q be the point of T farthest from these centers. For convenience, we rename $q = c_{k+1}$ (in fact, q would be the $(k+1)$ -st center selected by Farthest-First Traversal). Let also $C_1^*, C_2^*, \dots, C_k^*$ denote the k optimal clusters, induced by a set S^* of optimal centers, so that $\Phi_{\text{kcenter}}(P, S^*) = \Phi_{\text{kcenter}}^{\text{opt}}(P, k)$. By the pigeonhole principle there must exist two points c_a, c_b , with $1 \leq a < b \leq k+1$, that fall in the same optimal cluster C_i^* , for some i , which implies that $d(c_a, c_b) \leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$. We prove (B) through the following sequence of inequalities. For each $y \in T$:

$$\begin{aligned} d(y, S) &= d(y, \{c_1, c_2, \dots, c_k\}) \\ &\leq d(q, \{c_1, c_2, \dots, c_k\}) \\ &\leq d(q, \{c_1, c_2, \dots, c_{b-1}\}) \\ &\leq d(c_b, \{c_1, c_2, \dots, c_{b-1}\}) \quad (\text{by the choice of } c_b \text{ made by FFT}) \\ &\leq d(c_b, c_a) \quad (\text{since } c_a \in \{c_1, c_2, \dots, c_{b-1}\}) \\ &\leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k). \end{aligned}$$

Therefore, for each $y \in T$ there exists $c \in S$ such that $d(y, c) \leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$. $\square$

Exercise 5 Let P be a set of points in a metric space (M, d) , and let $T \subseteq P$. For any $k < |T|$, show that $\Phi_{\text{kcenter}}^{\text{opt}}(T, k) \leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$. Is the bound tight?

Solution. Let $(C_1, C_2, \dots, C_k, c_1, c_2, \dots, c_k)$ be the optimal k -clustering for P (i.e., $\Phi_{\text{kcenter}}(P, \{c_1, c_2, \dots, c_k\}) = \Phi_{\text{kcenter}}^{\text{opt}}(P, k)$). From each cluster C_i , remove the points not in T and pick one arbitrary remaining point (if any) as a new center. Let $S \subseteq T$ be the centers that have been picked in this fashion. (If $|S| < k$ add arbitrary points of T to make $|S| = k$.) This is pictorially represented in Figures 1, 2, and 3.

Figure 1: Optimal clustering of P Figure 2: Subset T

We have that

- For each $y \in T$ there must be $c \in S$ such that y and c belong to the same optimal cluster C_i .
- For any x, z in the same optimal cluster C_i , $d(x, z) \leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$ (by triangulating with the center c_i of the cluster)

Therefore, for each $y \in T$, $d(y, S) \leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k)$, which implies:

$$\Phi_{\text{kcenter}}^{\text{opt}}(T, k) \leq \Phi_{\text{kcenter}}(T, S) \leq 2\Phi_{\text{kcenter}}^{\text{opt}}(P, k).^1$$

Regarding how tight the upper bound is, consider the following example. Let

$$P = \{-1, 0, 1, x, 2x, \dots, (k-1)x\}$$

be a set of $k+2$ points in $\mathbb{R}$, for some large $x > 1$, and let

$$T = \{-1, 1, x, 2x, \dots, (k-1)x\} = P - \{0\}.$$

Convince yourself that, for large x , $\Phi_{\text{kcenter}}^{\text{opt}}(P, k) = 1$ and $\Phi_{\text{kcenter}}^{\text{opt}}(T, k) = 2$. □

¹Alternatively, we could have computed S using Farther-First Traversal, reaching the same conclusions by reasoning as in the previous exercise.

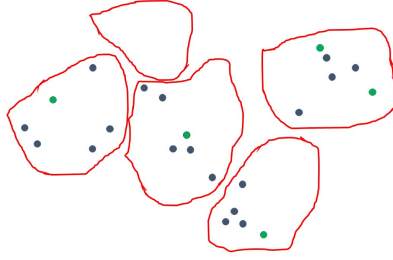


Figure 3: Coreset (green points)

Exercise 6 Consider a set P of N points from a metric space, represented by the key-value pairs (i, x_i) , with $0 \leq i < N$ and $x_i \in P$. Let S be a set of k centers from the same metric space but not belonging to P , which are stored in a global array $S[1], S[2], \dots, S[k]$. For $\ell = 1, 2, \dots, k$, denote with $P^{(\ell)}$ the set of points of P whose closest center is $S[\ell]$. Assume that each point in P has exactly one closest center in S , and that $k = O(\sqrt{N})$.

1. Design a 2-round MapReduce algorithm that, given P and S , computes a coreset $T \subset P$ containing one point from each non-empty $P^{(\ell)}$.
2. Analyze the local and aggregate space required by your algorithm. For full score, your algorithm must require $o(N)$ local space and linear aggregate space.
3. Show how to use T to approximate the value $\Delta(P) = \max_{x, y \in P} d(x, y)$, and estimate the quality of the approximation in terms of $R = \max_{x \in P} d(x, S)$. (Recall that each center $S[\ell]$ is not in P , hence it cannot be in T .)

Solution.

1. The MapReduce algorithm is the following.

Round 1:

- *Map phase*: map each pair (i, x_i) , into the intermediate pair $(i \bmod \sqrt{N}, x_i)$.
- *Reduce phase*: for each $0 \leq j < \sqrt{N}$, let L_j be the list of values x from the intermediate pairs with key j . For each center $S[\ell]$, with $1 \leq \ell \leq k$, select on point $x \in L_j \cap P^{(\ell)}$, if any, and output the pair (ℓ, x) . Hence, at most ℓ pairs are output from L_j .

Round 2:

- *Map phase*: empty.
- *Reduce phase*: for each key ℓ , let L_ℓ be the list of values x from the pairs with key ℓ produced by Round 1, and return one arbitrary such value.

2. In the map phase of Round 1, each application of the map function requires constant space; in the reduce phase of Round 1 each reducer processes at most $\sqrt{N}$ elements and needs $O(K) = O(\sqrt{N})$ space to store the centers; in the reduce phase of Round 2 each reducer processes at most $\sqrt{N}$ elements. Therefore, the local space is $M_L = O(\sqrt{N})$. The aggregate space is $O(N)$ since input, output and intermediate pairs in each round are $O(N)$.
3. We can approximate $\Delta(P)$ with $\Delta(T) = \max_{x,y \in T} d(x,y)$. Clearly, $\Delta(P) > \Delta(T)$. An upper bound to $\Delta(P)$ can be computed as follows. Let $\Delta(P) = d(z,w)$ for some $z, w \in P$, and let $z \in P^{(\ell_1)}$ and $w \in P^{(\ell_2)}$ for some indices $\ell_1, \ell_2 \in [1, k]$. Clearly, $P^{(\ell_1)}$ and $P^{(\ell_2)}$ are non-empty, so the coreset T will contain a point $t_{\ell_1} \in P^{(\ell_1)}$ and a point $t_{\ell_2} \in P^{(\ell_2)}$. Note that $d(t_{\ell_1}, t_{\ell_2}) \leq \Delta(T)$ and that, by the triangle inequality, $d(z, t_{\ell_1}) \leq d(z, S[\ell_1]) + d(S[\ell_1], t_{\ell_1}) \leq 2R$ and $d(w, t_{\ell_2}) \leq d(w, S[\ell_2]) + d(S[\ell_2], t_{\ell_2}) \leq 2R$. By repeatedly using the triangle inequality, we easily obtain

$$\Delta(P) = d(z, w) \leq d(z, t_{\ell_1}) + d(t_{\ell_1}, t_{\ell_2}) + d(t_{\ell_2}, w) \leq d(t_{\ell_1}, t_{\ell_2}) + 4R < \Delta(T) + 4R.$$

By putting all pieces together we get that

$$\Delta(T) \leq \Delta(P) \leq \Delta(T) + 4R.$$

□

Exercise 7 Design an MR algorithm to compute the exact diameter of a set P of N points, which requires $R = O(1)$, $M_L = O(\sqrt{N})$ and $M_A = O(N^2)$. Assume that P is initially provided as the set of pairs (i, x_i) , for $0 \leq i < N$, where the x_i 's are the points.

Solution. The input-output specification of the problem is the following one:

- **Input:** Set P of N points represented by pairs (i, x_i) , for $0 \leq i < N$.
- **Output:** $(0, \Delta(P) = \max_{0 \leq i, j < N} d(x_i, x_j))$.

The idea is first to compute the distances between all possible pairs x_i, x_j with $0 \leq i \leq j < N$ (this is done in the first 2 rounds), and then to compute the maximum among them in several rounds, processing distances in groups of size $\sqrt{N}$. The details of the algorithm are below.

Round 1

- *Map phase:* map each pair (i, x_i) into the following $\sqrt{N}$ intermediate pairs: $((i, j), x_i)$, with $0 \leq j < \sqrt{N}$.

Reduce phase: empty

Round 2

- *Map phase*: map each pair $((i, j), x_i)$ into the following $\sqrt{N}$ intermediate pairs: $((i, j * \sqrt{N} + j'), x_i)$, with $0 \leq j' < \sqrt{N}$.

Reduce phase: empty

Observation: At this point, for every x_i we have N pairs $((i, j), x_i)$, for $0 \leq j < N$.

Round 3

- *Map phase*: map each pair $((i, j), x_i)$ into the pair $((\min\{i, j\}, \max\{i, j\}), x_i)$.
- *Reduce phase*: for every $0 \leq i \leq j < N$, let $L_{i,j}$ be the list containing the values of the two pairs with key (i, j) , namely x_i and x_j , and return pair $(i * N + j, d(x_i, x_j))$.

Observation: At this point we have all pairwise distances among points, represented in $N(N+1)/2 < N^2$ pairs with distinct integer keys in $[0, N^2)$. (Obs. Rounds 2 and 3 can be easily merged into one round, but we kept them separate for clarity.)

Round 4

- *Map phase*: map each pair (k, d) into the intermediate pair $(k' = k \bmod N^{3/2}, d)$.
- *Reduce phase*: for each $k' \in [0, N^{3/2})$, let $L_{k'}$ be the list of values (i.e., distances) from intermediate pairs with key k' , and return pair $(k', d' = \max\{d \in L_{k'}\})$. Note that $|L_{k'}| \leq \sqrt{N}$.

Round 5

- *Map phase*: map each pair (k, d) into the intermediate pair $(k' = k \bmod N, d)$.
- *Reduce phase*: for each $k' \in [0, N)$, let $L_{k'}$ be the list of values (i.e., distances) from intermediate pairs with key k' , and return pair and return pair $(k', d' = \max\{d \in L_{k'}\})$. Note that $|L_{k'}| \leq \sqrt{N}$.

Round 6

- *Map phase*: map each pair (k, d) into the intermediate pair $(k' = k \bmod \sqrt{N}, d)$.
- *Reduce phase*: for each $k' \in [0, \sqrt{N})$, let $L_{k'}$ be the list of values (i.e., distances) from intermediate pairs with key k' , and return pair and return pair $(0, d' = \max\{d \in L_{k'}\})$. Note that $|L_{k'}| \leq \sqrt{N}$.

Round 7

- *Map phase*: empty.
- *Reduce phase*: let L_0 be the list of values of all pairs produced by Round 6, into pair $(0, L)$, where L is the list of values from these pairs, and return $(0, \max_{d \in L} d)$ as final output.

It is easy to see that the above 7-round algorithm uses aggregate space proportional to the number of ordered pairs of points, that is, $M_A = O(N^2)$. Let us analyze the local space. Each map phase of Rounds 1 and 2 transforms a single pair into $\sqrt{N}$ intermediate pairs, thus requiring $O(\sqrt{N})$ local space, while for the map phases of the other rounds $O(1)$ local space is sufficient. Each non-empty reduce phase works separately on lists of at most $O(\sqrt{N})$ values and produces one pair out of each list, thus requiring $O(\sqrt{N})$ local space. For the above considerations it follows that $M_L = O(\sqrt{N})$. $\square$

Exercise 8 Let P be a set of N points from metric space (M, d) . For an arbitrary $x \in P$ show that

$$\Delta_x = \max\{d(x, y) : y \in P\},$$

(which is a 2-approximation to the diameter) can be computed in MapReduce using 2 rounds, $M_L = O(\sqrt{N})$ and $M_A = O(N)$.

Solution. We use the following input-output specification for the algorithm:

- **Input:** $P = \{(j, x_j) : 0 \leq j < N\}$ and index $i \in [0, N)$, assumed to be a global variable.
- **Output:** $(0, \Delta_i)$.

The algorithm is the following.

Round 1.

- *Map phase:* map each pair (j, x_j) , with $j \neq i$, into the intermediate pair $(j \bmod \sqrt{N}, (j, x_j))$, and map pair (i, x_i) into the set $\{(h, (i, x_i)) : 0 \leq h < \sqrt{N}\}$ of $\sqrt{N}$ pairs.
- *Reduce phase:* for each $h \in [0, \sqrt{N})$, let L_h be the list of values (j, x_j) from intermediate pairs with key h (note that L_h includes also (i, x_i)), and return the pair $(0, \max_{(j, x_j) \in L_h} d(x_i, x_j))$.

Round 2.

- *Map phase:* empty.
- *Reduce phase:* let L_0 be the list of the values (i.e., partial maximum distances from x_i) of the pairs produced by Round 1, and return pair $(0, \Delta_i = \max\{d \in L\})$ as final output.

For what concerns the local space, the map phase of Round 1 requires $O(\sqrt{N})$ space to produce the copies of (i, x_i) , while the reduce phases of both rounds process lists of size $O(\sqrt{N})$. Therefore, $M_L = O(\sqrt{N})$. As for aggregate space, the number of input, output and intermediate pairs in each round is $O(N)$, hence the aggregate space is linear in the input size. $\square$

Exercise 9 Recall that *k-means++* is a randomized algorithm and the quality of the solution is a random variable. Letting S be the set of centers returned by *k-means++* for P it is known that there is a value $\alpha = \Theta(\ln k)$ such that

$$\Pr(\Phi_{\text{kmeans}}(P, S) \leq \alpha \cdot \Phi_{\text{kmeans}}^{\text{opt}}(P, k)) \geq 1/2.$$

Show that the above probability can be made $\geq 1 - 1/N$, if we run several independent instances of *k-means++*, and return the set of centers which yields the minimum value of the objective function, among the ones computed in the various runs.

Solution. Suppose that we execute t independent runs of *k-means++* on P , and let S_i be the set of centers returned by the i -th run, for $1 \leq i \leq t$. For each i , observe that each $\Phi_{\text{kmeans}}(P, S_i)$ is a random variable and define the event $E_i = \Phi_{\text{kmeans}}(P, S_i) > \alpha \cdot \Phi_{\text{kmeans}}^{\text{opt}}(P, k)$. Clearly, these events are mutually independent and, by hypothesis, $\Pr(E_i) \leq 1/2$ for every i . Define $E = \bigcap_{i=1}^t E_i$, and $S_{\min}$ as the set of centers, among the S_i 's, which minimizes the value of the objective function, namely,

$$S_{\min} = \operatorname{argmin}_{S_i: 1 \leq i \leq t} \Phi_{\text{kmeans}}(P, S_i).$$

It is immediate to see that event E occurs iff $\Phi_{\text{kmeans}}(P, S_{\min}) > \alpha \cdot \Phi_{\text{kmeans}}^{\text{opt}}(P, k)$, and that:

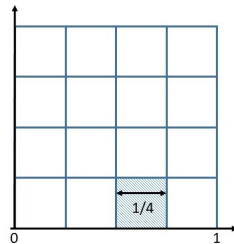
$$\begin{aligned} \Pr(E) &= \prod_{i=1}^t \Pr(E_i) \\ &\leq 1/2^t \quad (\text{by independence of the } E_i \text{'s}). \end{aligned}$$

By setting $t = \log_2 N$, we obtain that $\Pr(E) \leq 1/N$, hence,

$$\begin{aligned} \Pr(\Phi_{\text{kmeans}}(P, S_{\min}) \leq \alpha \cdot \Phi_{\text{kmeans}}^{\text{opt}}(P, k)) &= 1 - \Pr(E) \\ &\geq 1 - 1/N, \end{aligned}$$

as required. $\square$

Exercise 10 Let Q be the unit square in $\mathbb{R}^2$, with bottom-left corner $(0,0)$, and consider Q subdivided into c^2 square cells of side $1/c$. Below is an example for $c = 4$.



Consider a set of points P from Q and a subset $T \subset P$ of at most c^2 points, which contains one point from each cell (if one exists). Let $\Delta(P)$ be the diameter of P and $\Delta(T)$ the diameter of T . Quantify how well $\Delta(T)$ approximates $\Delta(P)$.

Solution. Since $T \subseteq P$, we clearly have that

$$\Delta(T) \leq \Delta(P).$$

To provide a lower bound to $\Delta(T)$ we reason as follows (see on the Coreset Technique (Part 2)). Let z, w be the two points of P such that $d(z, w) = \Delta(P)$, and let a and b be the points in T belonging to the same cells where z and w reside, respectively. Let R be the maximum distance between two points in the same cell (i.e., $R \leq \sqrt{2}/c$). We have $d(z, a), d(w, b) \leq R$. Also, since $a, b \in T$, we have $d(a, b) \leq \Delta(T)$. Now, by repeatedly applying the triangle inequality, we obtain

$$\Delta(P) = d(z, w) \leq d(z, a) + d(a, b) + d(b, w) \leq \Delta(T) + 2R \leq \Delta(T) + 2\sqrt{2}/c,$$

therefore,

$$\Delta(T) \geq \Delta(P) - 2\sqrt{2}/c.$$

□

Exercise 11 Let P be a set of N points in a metric space (M, d) , and let $\mathcal{C} = (C_1, C_2, \dots, C_k; c_1, c_2, \dots, c_k)$ be a k -clustering of P . Initially, each point $q \in P$ is represented by a pair $(ID(q), (q, c(q)))$, where $ID(q)$ is a distinct key in $[0, N - 1]$ and $c(q) \in \{c_1, \dots, c_k\}$ is the center of the cluster of q .

1. Design a 2-round MapReduce algorithm that for each cluster center c_i determines the most distant point among those belonging to the cluster C_i (ties can be broken arbitrarily).
2. Analyze the local and aggregate space required by your algorithm. Your algorithm must require $o(N)$ local space and $O(N)$ aggregate space.

Solution. The algorithm is the following:

Round 1

- *Map phase:* each pair $(ID(q), (q, c(q)))$ is mapped into the intermediate pair $(j, (q, c(q)))$, with $j = ID(q) \bmod \sqrt{N}$.
- *Reduce phase:* For each $0 \leq j < \sqrt{N}$, let L_j be the list of values $(q, c(q))$ from intermediate pairs with key j . For each center c occurring in L_j determine the most distant point q such that $(q, c(q) = c) \in L_j$, and return the pair $(c(q), q)$, where $c(q)$ is the key. (**Observation:** At the end of the round there will be at most $\sqrt{N}$ pairs with the same key.)

Round 2

- *Map phase*: empty.
- *Reduce phase*: for each center c , let L_c be the list of values (i.e., points) from the pairs produced by Round 1 with key c , and return pair (q_{max}, c) where $q_{max} = \operatorname{argmax}_{q \in L_c} d(c, q)$.

Correctness immediately follows by the observation that for each center c , if the most distant point is q and $(q, c(q) = c) \in L_j$, the pair $(c(q), q)$ will be returned at the end of the first round by the reducer processing L_j , hence q will be identified as the most distant point from c in the second round. Since each L_j has size $\Theta(\sqrt{N})$ and, in the first round, at most $\sqrt{N}$ points are selected for each cluster, the local space is $M_L = \Theta(\sqrt{N})$. Also, it is easy to see that input, output and intermediate pairs in each round are $O(N)$, hence the aggregate space linear in N . $\square$

Exercise 12 Let P be a set of N bicolored points from a metric space, partitioned into k clusters $C_1, C_2, \dots, C_k$. Each point $x \in P$ is initially represented by the key-value pair $(ID_x, (x, i_x, \gamma_x))$, where ID_x is a distinct key in $[0, N - 1]$, i_x is the index of the cluster which x belongs to, and $\gamma_x \in \{0, 1\}$ is the color of x .

1. Design an efficient 2-round MapReduce algorithm that for each cluster C_i checks whether all points of C_i have the same color. The output of the algorithm must be the k pairs (i, b_i) , with $1 \leq i \leq k$, where $b_i = -1$ if C_i contains points of different colors, otherwise b_i is the color common to all points of C_i .
2. Analyze the local and aggregate space required by your algorithm. Your algorithm must require $o(N)$ local space and $O(N)$ aggregate space.

Solution.

1. The 2-round MapReduce algorithm is the following.

Round 1:

- *Map phase*: map each pair $(ID_x, (x, i_x, \gamma_x))$, into the intermediate pair $(ID_x \bmod \sqrt{N}, (x, i_x, \gamma_x))$.
- *Reduce phase*: for each $0 \leq j < \sqrt{N}$, let L_j is the list of values from the intermediate pairs with key j . For each cluster index i such that there exists $(x, i_x = i, \gamma_x) \in L_j$, return the pair $(i, b_i(j))$, where $b_i(j) = -1$, if L_j contains points from cluster C_i of different colors, otherwise $b_i(j)$ is the color common to all points of C_i in L_j .

Round 2:

- *Map phase*: empty.
 - *Reduce phase*: for each $1 \leq i \leq k$, let L_i be the list of values $b_i(j)$'s, with $j \in [0, \sqrt{N})$, from the pairs with key i produced by Round 1, and return (i, b_i) where $b_i = 0$, if all $b_i(j)$'s in L_i are 0, $b_i = 1$, if all $b_i(j)$'s in L_i are 1, and $b_i = -1$ in all other cases.
2. The local space is $M_L = O(\sqrt{N})$ since the map phase of Round 1 uses constant space, while the reduce phases of both rounds process at most $\sqrt{N}$ elements each. The aggregate space is $O(N)$ since input, output and intermediate pairs in each round are $O(N)$.

□

Exercise 13 Let P be a set of N weighted points partitioned into k clusters $C_1, C_2, \dots, C_k$ with respective centers $c_1, c_2, \dots, c_k$. Each point $x \in P$ is represented by the pair $(x, (i_x, w_x))$, where i_x is the index of the cluster of x , and w_x is the weight of x . For every cluster C_i its weighted closest neighbor is the cluster C_j , with $j \neq i$, which minimizes the following sum

$$\sum_{x \in C_i} w_x \cdot d^2(x, c_j) \quad (c_j \text{ center of } C_j).$$

1. Design an efficient 2-round MapReduce algorithm that for each $1 \leq i \leq k$ computes the index j of C_i 's closest neighbor. You can assume k constant and that $c_1, \dots, c_k$ to be global variables.
2. Analyze the local and aggregate space required by your algorithm. For full score, your algorithm must require $o(N)$ local space and $O(N)$ aggregate space.

Solution.

1. The input-output specification of the algorithm is the following:

- **Input:** $P = \{(x, (i_x, w_x)) : x \in P, \text{ index } i_x, \text{ weight } w_x\}$
- **Output:** $\{(i, j_i) : 1 \leq i \leq k\}$, where j_i denotes the index of C_i 's weighted closest cluster.

The algorithm is the following.

Round 1.

- *Map phase*: map each pair $(x, (i_x, w_x))$ into the intermediate pair $(r_x, (x, i_x, w_x))$, where r_x is a random integer in $[0, \sqrt{N})$.
- *Reduce phase*: for each $h \in [0, \sqrt{N})$, let L_h be the list of values (x, i_x, w_x) from intermediate pairs with key h , and for every cluster index i occurring in L_h produce in output the $k - 1$ pairs $(i, (j, \text{sum}_h(i, j)))$, for $1 \leq j \neq i \leq k$, where

$$\text{sum}_h(i, j) = \sum_{(x, i_x=i, w_x) \in L_h} w_x \cdot d^2(x, c_j).$$

Round 2.

- *Map phase*: empty.
 - *Reduce phase*: for $1 \leq i \leq k$ let L_i be the list of values $(j, \text{sum}_h(i, j))$ from the output pairs of Round 1, and return the pair (i, j_i) , where j_i is the index j which minimizes $\sum_{(j, \text{sum}_h(i, j)) \in L_i} \text{sum}_h(i, j)$.
2. Recall that we assumed k to be a constant. For what concerns the local space: the map phase of Round 1 requires $O(1)$ space, the reduce phase of Round 1 requires $O(\sqrt{N})$ space with high probability (same analysis done in class for the 2-round class count algorithm with random partitions), and the reduce phase of Round 2 requires $O(\sqrt{N})$ space. Overall, we have $M_L = O(\sqrt{N})$ with high probability. As for the aggregate space, the number of input, output and intermediate pairs in each round is $O(N)$, hence the aggregate space is linear in the input size.

□